

# Analysis: Ambiguous Cipher

## Ambiguous Cipher: Analysis

### Small dataset

There are only  $26^2 + 26^3 + 26^4 = 475228$  words with lengths in the  $[2, 4]$  range. The statement tells us exactly how to encrypt a word, so we can encrypt all possible words and then create a reverse dictionary. For example, when we find that `SOUP` encrypts to `OMDU`, we can create an entry in our reverse dictionary with key `OMDU` and value `SOUP`. If we ever find that a word has encrypted to a key we already have in the dictionary (for example, `ABC` and `CBA` both encrypt to `BCB`), then that key's decryption is ambiguous, and we can replace the value with `AMBIGUOUS`.

Once we have done this, solving the problem is easy; we just look up each word in our dictionary and output the value as our answer. We do not need to worry about words with no valid decryption (and thus no dictionary key), because the problem statement guarantees that these will not appear in the datasets.

This method will not work for the Large dataset, though; in that case, there are just too many possible words to stuff into a dictionary in memory! We need another approach.

### Large dataset: words with even lengths

For now, let's assume our encrypted word has an even length. We will consider words of odd length in the next section.

When a word is being encrypted, every letter in the word (except for the first and last) is encrypted by summing the two letters adjacent to that letter. For the first and last letters, the encrypted value is just the unencrypted value of the one letter adjacent to it. This means that we can easily find the the second and second-to-last letters of the unencrypted word: they are just the first and last letters of the encrypted word, respectively.

Moreover, once we have these "footholds" into the decrypted word, we can use them to decrypt the rest of the encrypted word! For example, we can decrypt the encrypted word `ADMZZO` as follows. In this explanation, we will denote a letter with E or D (referring to the encrypted or decrypted words) followed by a number (giving the letter's position in the word, counting starting from 1.)

- The letter values of the encrypted word are: 0 3 12 25 25 14.
- We know that D2 and D5 equal E1 and E6, as explained above, so we know that our decrypted word has letter values `_ 0 _ _ 14 _`.
- We will now determine the letters in the even-numbered positions of the decrypted word, going from left to right. We will start by determining D4. We already know that D2 has value 0. Since we know that  $(D2 + D4) \% 26 = E3$ , then  $D4 = (E3 - D2) \% 26$ . E3 has value 12, so  $D4 = (12 - 0) \% 26 = 12$ . So now our decrypted letter values are: `_ 0 _ 12 14 _`.
- Now that we know D4, we can find D6 in the same way:  $D6 = (E5 - D4) \% 26 = (25 - 12) \% 26 = 13$ . So we have `_ 0 _ 12 14 13 _`.
- Then, we do the same thing from right to left, starting with D5. We have  $D3 = (E4 - D5) \% 26 = (25 - 14) \% 26 = 11$ , and  $D1 = (E2 - D3) \% 26 = (3 - 11) \% 26 = 18$ . (It is OK to take the modulus of a negative number, but we can also always safely add 26 if we want to only

handle positive values, e.g.,  $(3 - 11 + 26) \% 26$ .) So we know all the letter values in the decrypted word: 18 0 11 12 14 13.

- Now we just need to convert this list of decrypted letter values back to uppercase letters, which results in the decrypted word `SALMON`.

This method will allow us to decrypt any word with an even length. Because the method was deterministic and we had no choices about how to decrypt, we can also see that any encrypted word with an even length has exactly one possible decryption; that is, in mathematical terms, decryption is both one-to-one and onto. But what about words with odd lengths?

## Large dataset: words with odd lengths

Let's look back at the example `BCB` from the problem statement. We know that  $D2 = E1$  and  $D2 = E3$ , so  $D2$  is `B`. But what about  $D1$  and  $D3$ ? All we know is that  $(D1 + D3) \% 26 = E2$ . There are 26 different pairs for which this is valid! So the decryption is ambiguous.

In fact, this is the case for *any* word of odd length that could appear in our datasets. Our method above does not apply! Knowing the second and second-to-last letters of the decrypted word does not help us get the entire word, since now both of those letters are in even-numbered positions of the decrypted word; they can only get us letters in the even-numbered positions of the decrypted word. We have no foothold that we can use to get any of the letters in the odd-numbered positions. The first letter of the decrypted word could be any letter, and any choice determines all the others. We have no way of knowing which of the 26 possibilities Calvin originally encrypted.

The statement tells us that the datasets do not include words like `APE` which have no valid decryption, so any word of odd length in our dataset must have multiple decryptions. With this knowledge, we can finish our solution: for words of even length, we use the method above, and for words of odd length, we can just report `AMBIGUOUS`. Calvin and Susie should come up with a method that can handle all words, not just words of even length!

# Analysis: Magical Thinking v2

## Magical Thinking: Analysis

### Small dataset

In the Small dataset, there is only one other friend to consider, and we know how many questions  $S_0$  they got right. We need to choose exactly  $S_0$  of the questions to be the ones they got right (call this set R); that means they got the other  $Q - S_0$  questions wrong (call this set W). Moreover, we need to do this in the way that maximizes our own score.

Let's divide the questions into two types: type A when we gave the same answer as our friend, or type D when we gave different answers. Suppose that we assign the questions to sets R and W in some way, and suppose that there is a type D question in set R, and a type A question in set W. Then we got both of those questions wrong! But if we swap the two, then we instead got both questions right. So it is always advantageous to make such swaps, and we should always load up set R with as many type A questions as we possibly can. If there are not enough type A questions to do this, then we must fill up the rest of set R with type D questions.

We can either implement this greedy strategy, or generalize it into a formula: if X is the number of questions on which we agree with our friend, then the answer turns out to be  $Q - |S_0 - X|$ .

Another possible approach for the Small dataset is to use brute force. Even in the worst case of  $Q = 10$ , there are only  $2^{10} = 1024$  different possible sets of answers. We can check all of them and see which ones are consistent with our friend's score, and which of those gives us the highest score.

### Large dataset

The Large dataset introduces two additional complications. For one thing, there can now be up to 50 questions, so the brute force approach mentioned above won't work;  $2^{50}$  is over  $10^{15}$ , and there are far too many possible sets of answers to check.

Our greedy method above will work for  $N = 1$ ,  $Q = 50$ , but adding a second friend introduces some complications. For example, trying to make greedy decisions about one friend in the same way we did for Small dataset 1 might cause the other friend to end up with the wrong score. However, we don't need a greedy approach, because we can take advantage of the small number of types of questions. When  $N = 2$ , there are only four types:

1. Both friends agree with us.
2. Only friend 1 agrees with us.
3. Only friend 2 agrees with us.
4. Neither friend agrees with us.

We can try every possible tuple (a, b, c, d) of the numbers of Type 1, 2, 3, and 4 questions that we got right, with  $0 \leq a \leq$  the number of Type 1 questions,  $0 \leq b \leq$  the number of Type 2 questions, and so on. Any tuples that do not cause both friends to have the correct scores should be discarded. We can then choose the remaining tuple for which  $a + b + c + d$  is maximized. This method is  $O(Q^4)$ , but the greatest possible number of such tuples to check occurs when  $Q = 50$  and there are 12 or 13 of each of the four types, and even then, there only about 25000 possibilities to check.

Another option for this dataset is to use [dynamic programming](#). For example, we can start with a set containing only the list  $[0, 0, 0]$ , which represents that at the start of the test, before any questions are answered, we have 0 points, friend 1 has 0 points, and friend 2 has 0 points. Then, suppose that our answer to the first question is F, our first friend answered T, and our second friend answered F. There are two options: either the real answer to the first question is F (for which case we create the list  $[1, 0, 1]$ , since we and the second friend have earned a point, but the first friend has not), or the real answer is T (for which case we create the list  $[0, 1, 0]$ ). Then we replace our old set with the new set  $([1, 0, 1], [0, 1, 0])$ , and for each list in that set, we in turn consider what happens when we answer F or T for the second question, and so on. Once we have finished, we check for the list in which our friends' scores are both correct and we have the highest possible score.

Why is this method any better than brute force? The key difference is that if we create a list that is already in our set, we do not add another copy of it, so we do not waste time individually considering many nearly equivalent (for our purposes) answer sets. Because our scores and our friends' scores can range from 0 to  $Q$ , inclusive, there are  $(Q + 1)^{N + 1}$  possible lists; since (as an upper bound) we have to check all of them once for each question, the method is  $O(Q^{N + 2})$ . This is  $O(Q^4)$  when  $N = 2$ . In practice, this is a "slower  $O(Q^4)$ " than the method above, since we may need to check  $51^3 \times 50 = \text{over } 6 \times 10^6$  possibilities, but it is still easily fast enough to solve the Large dataset. The approach also works just fine with a three-dimensional array instead of a set, but the set lets us avoid checking for the existence of all possible lists at each step of the process.

# Analysis: The 4M Corporation

## The 4M Corporation: Analysis

### Small dataset

The Limits section of the problem statement tells us that the answer to every Small test case that is not `IMPOSSIBLE` is less than 14. This suggests that brute force is a viable option for the Small dataset. The tight bounds imposed by **MINIMUM** and **MAXIMUM** help us out as well. But let's think about the worst case: what if those two values are 1 and 8, respectively, and what if the answer ends up being 13? In that case, we know that two of the values in the list must be 1 and 8, but if any of the others can be any integer in  $[1, 8]$ , then we can have  $8^{11}$  — over 8 billion — possibilities for the remaining numbers. That's too much to handle in our limited time!

Fortunately, we can cut this daunting space of possibilities down to size by avoiding redundant lists. In this problem, we don't care about the difference between  $[1, 1, 2, 3]$  and  $[2, 1, 3, 1]$ , for instance. One way to avoid worrying about such differences is to only consider lists that are sorted in nondecreasing order. Even in the worst case mentioned above, there are only about 75,000 different nondecreasing lists that accord with the Small limits. (You can use a [combinatorial trick](#) to find this value, or you can just experiment and see.)

One simple way to actually generate those lists is to start with a list consisting of only **MINIMUM**, then generate all new legal lists that append one more value to the end of that, then generate all new legal lists that append one value to the ends of *those* lists, and so on. Since the number of such lists is small, an approach like this is tractable. Once you have all the lists, you can check each of them to see if it has the desired properties, and return the length of the shortest such list, or `IMPOSSIBLE` if none of the lists works.

### Large dataset

With values in the  $[1, 10000]$  range, and no upper bound on the answer, we will never be able to generate all possible lists in time. We need a general, non-brute-force solution.

Let's start with some special cases that might be easy to overlook:

- There are five sets of demands that make the task clearly `IMPOSSIBLE`, because they contradict the mathematical definitions of minimum, maximum, mean, and median:
  - **MAXIMUM** < **MINIMUM**
  - **MAXIMUM** < **MEDIAN**
  - **MAXIMUM** < **MEAN**
  - **MEDIAN** < **MINIMUM**
  - **MEAN** < **MINIMUM**
- The problem might be solvable with one department. This is possible if and only if **MINIMUM**, **MAXIMUM**, **MEAN**, and **MEDIAN** are all the same, so we can easily confirm or rule out this case.
- The problem might be solvable with two departments. This requires that **MEAN** and **MEDIAN** are both exactly  $(\text{MINIMUM} + \text{MAXIMUM}) / 2$ . If this is true, we have no reason to consider using more departments, so we are done.

Otherwise, suppose that the problem is solvable with three departments. Then those three values must be **MINIMUM**, **MEDIAN**, and **MAXIMUM**. Assuming that these values do not

already have the correct **MEAN**, perhaps we can insert more values until they do. We can repeatedly insert one value to the left of the median and one value to the right. First, we figure out how much the mean of the existing three values deviates from the desired mean. (To avoid errors associated with floating-point computation, it is better to compare the values' sum with **MEAN** times the number of values.) Suppose (without loss of generality) that the existing sum of values is too large. Then we can figure out which two values to insert to bring down this surplus as much as possible; usually, they will be another copy of **MINIMUM** (on the left) and another copy of **MEDIAN** (on the right). If inserting even one such pair won't reduce the surplus — that is, if **MEDIAN + MINIMUM** is no smaller than  $2 \times \text{MEAN}$  — then this method will never work. Otherwise, we can keep doing this until the surplus is gone. If inserting that last pair of **MINIMUM** and **MEDIAN** would overshoot the surplus by an amount  $K$ , we can just insert **MINIMUM + K** instead of the usual **MINIMUM**; this value will never exceed **MEDIAN**.) We don't even need to carry out each such step individually; we can figure out exactly how many will be needed.

Note that since we always insert one value to the left of our original **MEDIAN** and one to the right, the list always has a copy of **MEDIAN** in the right place. Also, given any other valid list of odd length, we can sort that list's values in nondecreasing order and pair them off (ignoring the first, last and middle elements, which are fixed): the second value with the second-to-last value, etc. Each of the pairs produced by our method contributes maximally towards reducing the surplus (except possibly the last one of value **MINIMUM + K**). So no valid list of odd length can be strictly shorter, as the total contribution would be strictly smaller.

We aren't done yet, though, because the above method only produces lists with an odd number of elements. So, regardless of what that method determined, we need to try something similar starting with four departments: **MINIMUM**, **MEDIAN**, **MEDIAN**, and **MAXIMUM**. (You may notice that we could also consider **MEDIAN - 1** and **MEDIAN + 1**, among other possible pairs, in place of **MEDIAN** and **MEDIAN**. But there is no reason to consider this, because it would only restrict the range of new values we can insert, and thus require more values to adjust the mean.) Once we get our answer from this four-department method, we can take the smaller of its answer and the three-department method's answer, or **IMPOSSIBLE** if neither method worked.

This solution runs in constant time. Our efficiency-loving bosses at the 4M Corporation will certainly be happy with that!

# Analysis: X Squared

## X Squared: Analysis

### Small dataset

One straightforward brute-force approach to this problem is to perform a [breadth-first search](#) of all grids that can be reached using the allowed swap operations, and see whether the target grid is reachable. However, this can take a while to code. Moreover, depending on the language used and the implementation details — e.g., how we choose to store previously seen states and avoid revisiting them — the search may run too slowly on the  $N = 5$  cases. Perhaps a breadth-first search is overkill here; can we identify all of the reachable states in advance, without exploring?

Let's think about what swap operations *cannot* change. The set of tiles in a particular row in the initial grid will always remain together, in some order, in some row (not necessarily the row they started in). This is because a column swap only changes the relative order of those tiles, and a row swap only moves that whole set of tiles around together. So, our operations on the board cannot exchange individual tiles among different rows; they can only change the internal and relative orders of existing rows.

Because column swaps are the only way to change the internal order within a row, changing one row to have a certain order will give all of the rows that same internal order. However, we can choose whichever internal order we want: we pick the column with the tile we want to be first, and swap that column into the first column position, and so on.

All of the above logic holds for columns as well, and we can operate on the columns independently enough for our purposes: if we reorder the rows, and then reorder the columns, the latter operations do not change the chosen order of the rows. So, we can choose any of the  $N!$  possible orders for our  $N$  rows, and any of the  $N!$  possible orders for our  $N$  columns, for a total of  $(N!)^2$  different states that we can reach. (Some of these states might have the same pattern of xes and blank tiles, even if they represent different rearrangements of the original tiles.) Unlike in our breadth-first search method, we don't need to worry about exactly how to reach each of these states, even though the end of the previous paragraph explains how to do so. We know that it is possible to reach all of them, so if one of them matches the target, the answer is `POSSIBLE`. Moreover, we know that these are all of the states that we can reach, so if none of them matches the target, the answer must be `IMPOSSIBLE`.

Now, all we have to do is write some code to permute the rows and columns of a grid into whatever orders we want. To avoid doing too much duplicate work, we can pick one row permutation, then apply all possible column permutations to copies of that, then pick another row permutation, and so on. This method is essentially a well-targeted [depth-first search](#), and its running time is  $O((N!)^2)$ . This is easily fast enough for the Small dataset, in which  $N$  maxes out at 5. But a squared-factorial order of growth won't work for the Large dataset! Let's look for a better method.

### Large dataset

Let's think about the target grid state. We can observe that it has one x tile that is the only x in its row and in its column; let's call this collection of x, row, and column the *singleton cross*. The rest of the grid contains a nested set of  $N/2$  of what we will call *rectangles*. Each rectangle is

defined by two rows and two columns, and has an  $\times$  tile at each of the four intersections of those rows/columns, and no other  $\times$  tiles anywhere else.

Suppose that we start at the target and perform some swap operations. What happens to our singleton cross and our rectangles? Per our earlier observation while solving the Small dataset, if two of the corners of a rectangle are together in the same row, or in the same column, they always will be; no row or column swaps can possibly split them apart. So swap operations cannot create or destroy rectangles, although they can change the relative positions of the four corners. Similarly, no operation can move additional  $\times$  tiles into the singleton cross, or take away its one  $\times$  tile.

These observations simplify the problem dramatically. We only need to ask: does the starting grid have exactly  $N/2$  rectangles and exactly one singleton cross? If not, there is no series of swaps that can turn it into the target, which does have those properties, so the case is `IMPOSSIBLE` and we are done. Otherwise, we can transform the starting grid into the target grid as follows. Choose a rectangle, and perform row and column swaps such that its corners end up as the outermost corners of the larger X shape. Then recursively solve the rest of the grid as a subproblem, and so on. At the end of all this, the singleton cross will necessarily be in the correct place, since every other place will have been taken.

With this reframing of the problem in mind, we no longer need to actually perform or even think about any swaps! We only need to check the starting grid for rectangles and a singleton row/column. It is possible to do this with a single pass through the grid, looking at each row in turn. If a row has zero or more than two  $\times$  tiles, it cannot be part of a rectangle or a singleton cross, and so the case is `IMPOSSIBLE`. If a row has one  $\times$  tile, we need it to be part of the singleton cross; we store the column location of that  $\times$ , and if we see another would-be singleton cross row later, the case must be `IMPOSSIBLE`. If a row has two  $\times$  tiles, we note the columns of those two tiles. Once we have checked all rows, we check to see that every two- $\times$  row's set of  $\times$  columns pairs up with exactly one other two- $\times$  row's set of  $\times$  columns, and that no such pair shares any position with any other pair. Moreover, the position of the  $\times$  in the singleton cross must be the sole column unused by any other pair. If all of these things are true, then the case is `POSSIBLE`.

This strategy is linear in the size of the input, and it runs fast enough to easily handle the Large dataset, as well as grids much larger than we provide in that dataset! It is also possible to run across greedy methods that carry out the swapping method we described earlier, or something that essentially boils down to it.